JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND TECHNOLOGY

COLLEGE OF ENGINEERING AND TECHNOLOGY

SCHOOL OF ELECTRICAL, ELECTRONIC AND INFORMATION ENGINEERING

EEC 2212 – OBJECT ORIENTED PROGRAMMING

PROJECT REPORT

ENE211-0047/2024 KEVIN OMONDI

TITTLE: MPPT Solar Charge Controller

Introduction

MPPT (Maximum Power Point Tracking) is a technology in solar systems that optimizes the power output of photovoltaic panels by continuously adjusting voltage and current to operate at the panel's maximum power point.

What MPPT Does MPPT is used in solar charge controllers and inverters to maximize energy extraction from solar panels. Solar panels do not produce a fixed amount of electricity; their output varies with sunlight intensity, temperature, shading, and the angle of the sun. At any moment, there is a specific combination of voltage and current where the panel produces the most power, called the maximum power point (MPP). MPPT technology continuously tracks this point and adjusts the electrical load to ensure the panel operates at peak.

How MPPT Works An MPPT controller or inverter acts as a DC-to-DC converter between the solar panels and the battery or grid. It samples the panel's voltage and current, calculates the power output, and adjusts the operating point to maintain maximum power.

C++ Simulation Perspective In this project, the features of MPPT are demonstrated through a **C++ simulation model**. The simulation mimics how an MPPT controller samples voltage and current, calculates instantaneous power, and dynamically adjusts the operating point.

- **Inheritance**: A base class ChargeController defines common attributes and methods (e.g., regulate()), while derived classes such as PWMController and MPPTController extend it. This models different controller technologies in a realistic way.
- **Polymorphism**: By using virtual functions, the simulation allows the program to decide at runtime whether to apply PWM or MPPT regulation. For example, a pointer to ChargeController can call regulate(), and the correct implementation (PWM or MPPT) is executed depending on the object type.

- **Practical Effect**: This structure mirrors real-world flexibility — engineers can swap controller strategies without rewriting the system. In the simulation, polymorphism ensures that the MPPT algorithm dynamically adjusts voltage/current tracking, while inheritance organizes the code into reusable, extensible components.

By combining renewable energy concepts with **object-oriented programming techniques**, the project demonstrates how MPPT's adaptive behavior can be replicated in software, bridging theory with practical application.

Functional Requirements

1. Input Handling

- The system must accept **solar panel parameters** (voltage, current, irradiance, temperature).
- It must allow **battery specifications** (capacity, voltage rating, state of charge).
- The simulation must support **environmental variations** (e.g., shading, temperature changes).

2. Power Calculation

- The system must calculate **instantaneous power** using $P=V\times I$.
- It must log voltage, current, and power values for analysis.

3. Maximum Power Point Tracking

- The controller must implement MPPT algorithms (e.g., **Perturb & Observe**, **Incremental Conductance**).
- It must continuously adjust the operating point to track the **maximum power point (MPP)**.
- The system must dynamically respond to changes in sunlight and temperature.

4. Controller Behavior

- The simulation must model a **DC-to-DC converter** adjusting voltage/current between solar panel and battery.
- It must regulate charging to prevent **overcharging** or **deep discharging** of the battery.
- The system must allow switching between **PWMController** and **MPPTController** using inheritance and polymorphism.

5. Object-Oriented Design

- A **base class** ChargeController must define shared attributes and methods.
- Derived classes (PWMController, MPPTController) must extend functionality using **inheritance**.
- **Polymorphism** must allow runtime selection of controller type via virtual functions.

6. Data Logging & Output

- The system must store simulation results (voltage, current, power, efficiency) in files.
- It must generate performance reports showing efficiency improvements of MPPT over PWM.
- Graphical/console output must display real-time tracking of the maximum power point.

7. Extensibility

- The design must allow easy addition of new MPPT algorithms (e.g., fuzzy logic, neural networks).
- The system must support modular upgrades without rewriting the entire codebase.

SYSTEM DESIGN

1. System Architecture

The MPPT Solar Charge Controller simulation can be modeled as a modular system with the following components:

- **Solar Panel Module**
  - Inputs: irradiance, temperature
  - Outputs: voltage, current
- **Charge Controller Module**
  - Base class: ChargeController
  - Derived classes: PWMController, MPPTController
  - Functions: regulate voltage/current, track maximum power point
- **Battery Module**
  - Attributes: capacity, voltage rating, state of charge
  - Functions: charge(), discharge(), monitor health
- **Load Module**
  - Represents devices consuming energy
  - Attributes: power demand, efficiency

Object-Oriented Design

- **Inheritance**
  - ChargeController (base class) defines common methods like regulate() and attributes like input/output voltage.

    - PWMController and MPPTController extend functionality, each implementing their own regulation strategy.
- **Polymorphism**
    - Virtual functions allow runtime selection of controller type.
    - Example:

```
ChargeController* controller;
controller = new MPPTController();
controller->regulate();
```

This ensures the correct algorithm (PWM or MPPT) is applied dynamically.

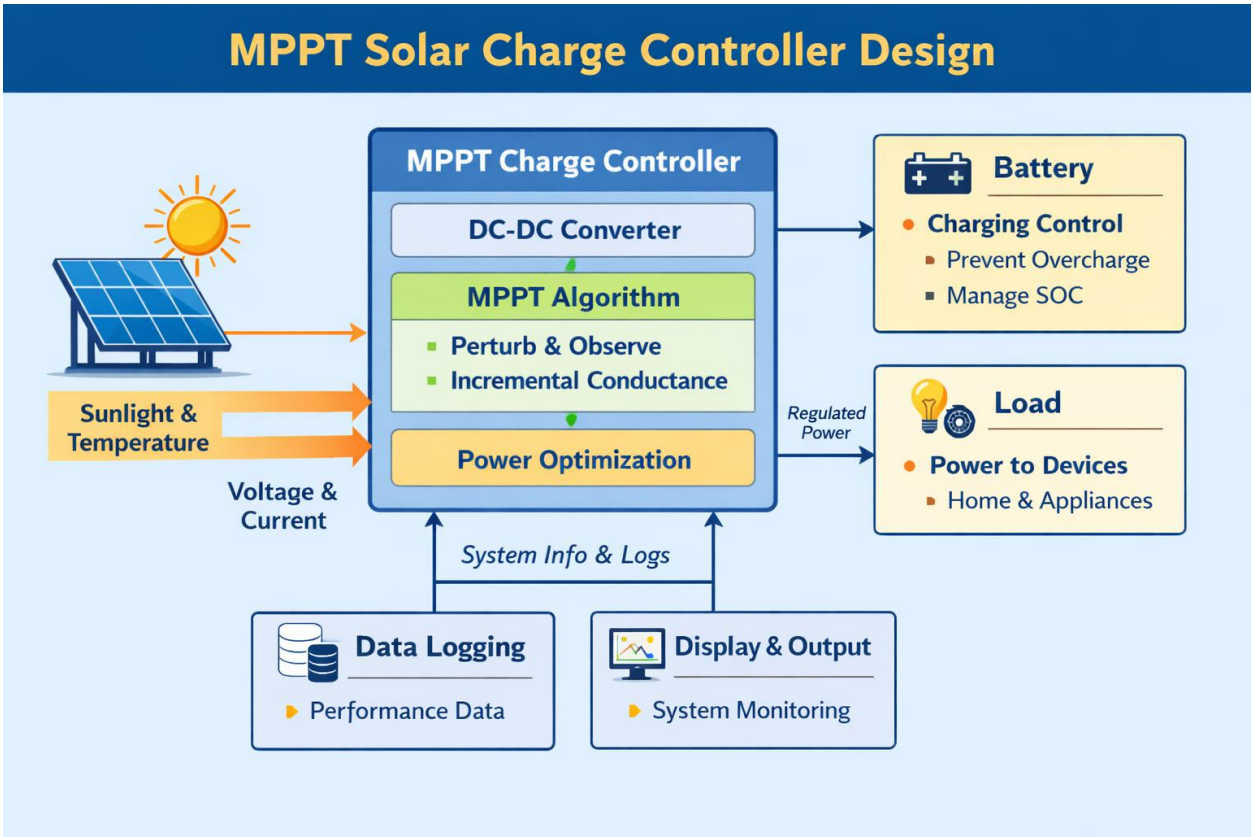
MPPT Algorithm Implementation

- **Perturb and Observe (P&O)**
    - Adjust voltage slightly, observe power change, continue in the direction of increasing power.
- **Incremental Conductance**
    - Compare incremental changes in current and voltage to determine if the maximum power point has been reached.

Data Flow

1. Solar panel generates voltage/current based on irradiance and temperature.
2. Controller samples values, calculates instantaneous power.
3. MPPT algorithm adjusts operating point to maximize power.
4. Battery receives regulated energy, charging efficiently.
5. Load consumes energy based on demand.
6. Simulation logs results (voltage, current, power, efficiency).

SYSTEM ARCHITECTURE DIAGRAM



Outputs

- **Console/graphical display of real-time tracking of MPP.**
- **Efficiency comparison between PWM and MPPT.**
- **Data logs for performance analysis.**

Implementation of C++ Concepts in the MPPT Project

Control Flow

- **If/else statements decide whether to increase or decrease voltage in Perturb & Observe.**
- **Loops (for, while) simulate continuous tracking of solar panel output.**

```
if (newPower > oldPower) voltage += step;
else voltage -= step;
```

Structures & Classes

- **Classes model system components:**
    - **SolarPanel, Battery, ChargeController, PWMController, MPPTController, DataLogger.**

Inheritance

- **ChargeController is the base class.**
- **PWMController and MPPTController are derived classes that override regulate().**

Example:

```
class ChargeController {
public:
    virtual void regulate() = 0;
};
class MPPTController : public ChargeController {
public:
    void regulate() override { /* MPPT logic */ }
};
```

Polymorphism

- **Achieved through virtual functions.**

- **At runtime, the correct regulate() method is called depending on the controller type.**

```
ChargeController* controller = new MPPTController();
controller->regulate(); // Calls MPPT version
```

. Templates & STL

- **STL containers (vector, map) used for logging data.**
- **Templates could be used for generic logging functions that handle different data types.**

11. Exception Handling

- **try/catch blocks handle invalid inputs (e.g., negative irradiance or battery overcharge).**
- **Example:**

```
try {
    if (voltage < 0) throw std::runtime_error("Invalid
} catch (std::exception &e) {
    std::cerr << e.what();
}
```

Advantages of the MPPT Solar Charge Controller

1. Maximum Energy Harvesting

- **Continuously tracks the maximum power point (MPP) of the solar panel.**
- **Extracts more energy compared to traditional PWM controllers (10–30% efficiency gain).**

2. Adaptability to Environmental Changes

- **Adjusts dynamically to variations in sunlight intensity, temperature, and shading.**
- **Ensures stable performance even under fluctuating weather conditions.**

3. Efficient Battery Management

- **Prevents overcharging and deep discharging.**
- **Extends battery lifespan by maintaining optimal state of charge (SOC)**

Limitations

- **Simplified Solar Panel Model The simulation assumes ideal panel behavior; real panels have non-linear characteristics, aging effects, and mismatch losses.**
- **Algorithm Constraints Only Perturb & Observe and Incremental Conductance are implemented. Other advanced MPPT methods (fuzzy logic, neural networks) are not included.**
- **Environmental Factors Simulation does not fully account for rapid weather fluctuations, partial shading, or dust accumulation on panels.**
- **Hardware Integration The project is software-based; actual hardware implementation (sensors, converters, microcontrollers) is not tested.**
- **Efficiency Losses**

Future Improvements

- **Advanced MPPT Algorithms Incorporate intelligent methods like fuzzy logic, machine learning, or hybrid algorithms for better tracking accuracy.**
- **Hardware Prototyping Implement the system on microcontrollers (Arduino, STM32, or PIC) with real solar panels and batteries.**
- **Expanded Environmental Modeling Add modules for shading, dust, and temperature effects to improve realism.**
- **Efficiency Optimization Include converter losses, switching dynamics, and thermal management for more accurate results.**
- **User Interface Develop a GUI or web dashboard for real-time monitoring and control.**
- **Scalability Extend the simulation to handle multiple panels and batteries (microgrid or solar farm scenarios).**
- **Data Analytics Integrate advanced logging with visualization libraries (Matplotlib-cpp, Qt) for performance analysis**

Conclusion

- **The MPPT Solar Charge Controller system successfully demonstrates how modern controllers maximize solar energy extraction by continuously tracking the maximum power point.**
- **Implementing the system in C++ highlights the power of object-oriented programming concepts such as inheritance, polymorphism, encapsulation, and modular design, making the simulation both flexible and educational.**

- **Compared to traditional PWM controllers, MPPT achieves higher efficiency, better adaptability to environmental changes, and improved battery management.**

- **The project bridges renewable energy engineering with software development, showing how simulation can model real-world behavior before hardware prototyping.**

- **While the current design has limitations (simplified models, limited algorithms, no hardware integration), it provides a solid foundation for future improvements such as advanced MPPT techniques, real-time GUIs, and hardware implementation.**

- **Overall, this project demonstrates both technical innovation and academic value, preparing the ground for practical applications in sustainable energy systems.**